

Trilha Banco de Dados SQL/NoSQL — Plano técnico até nível sênior

SQL / NoSQL
PostgreSQL
CDC
Redis
DynamoDB
Busca semântica

Material técnico estruturado como plano de estudos progressivo, com foco em SQL, NoSQL, PostgreSQL, CDC, Redis, DynamoDB, busca semântica e operação em produção.

Formato: apostila técnica de estudo e referência profissional

Objetivo do documento

Consolidar todo o conteúdo anexado em um PDF formatado, com leitura fluida, organização hierárquica e aparência profissional para estudo, revisão e consulta técnica.

Sumário

Lista das principais seções com numeração de páginas atualizada.

Trilha Banco de Dados SQL/NoSQL — Plano técnico até nível sênior	4
1. Análise inicial da trilha	4
2. Explicação técnica por grande área	5
2.1 SQL profissional	5
2.2 Modelagem relacional	8
2.3 Projeto prático relacional: banco da DevCore	10
2.4 PostgreSQL avançado	11
2.5 CDC e arquitetura orientada a eventos	14
2.6 MongoDB avançado	15
2.7 Redis avançado	17
2.8 DynamoDB	19
2.9 Vector databases e busca semântica	21
3. Organização como plano de estudos	23
Módulo 1 — Fundamentos de dados e SQL	23
Módulo 2 — SQL avançado e aplicação prática	24
Módulo 3 — PostgreSQL avançado e performance	25
Módulo 4 — Arquitetura de dados e eventos	27
Módulo 5 — NoSQL, cache, escalabilidade e busca	28
Módulo 6 — Domínio sênior, produção e liderança técnica	30
4. Critérios de domínio técnico por área	32
5. Aplicação prática em sistemas reais	33
6. Cronograma sugerido de execução	36
Checkpoint 1 — Semana 3	36
Checkpoint 2 — Semana 6	36
Checkpoint 3 — Semana 10	36
Checkpoint 4 — Semana 13	37
Checkpoint 5 — Semana 16	37
Checkpoint 6 — Semana 20	37
Checkpoint 7 — Semana 22	37
Checkpoint 8 — Semana 24	37
7. Projetos recomendados para portfólio	38

Projeto 1 — DevCore Database Platform	38
Projeto 2 — PostgreSQL Performance Lab	38
Projeto 3 — CDC Event Pipeline	38
Projeto 4 — Redis Performance Layer	39
Projeto 5 — Hybrid Semantic Search	39
8. Checklist de domínio técnico	39
9. Indicadores de prontidão para nível avançado/sênior	41
10. Resumo final da trilha	42

Trilha Banco de Dados SQL/NoSQL — Plano técnico até nível sênior

1. Análise inicial da trilha

Tema central

A trilha tem como tema central domínio profissional de dados em sistemas reais, cobrindo bancos relacionais, bancos NoSQL, cache, mensageria orientada a eventos, busca vetorial, recuperação, performance, segurança, escalabilidade e operação em produção.

O foco não é apenas “saber consultar banco”, mas desenvolver a capacidade de modelar, operar, otimizar, escalar e proteger dados em ambientes críticos.

Objetivo principal

Formar um profissional capaz de:

- Projetar modelos relacionais e NoSQL coerentes com o domínio.
- Escrever SQL avançado e otimizar queries lentas.
- Operar PostgreSQL em produção.
- Integrar banco de dados com arquitetura orientada a eventos.
- Escolher corretamente entre PostgreSQL, MongoDB, Redis, DynamoDB e bancos vetoriais.
- Tomar decisões arquiteturais justificáveis sobre consistência, custo, performance, escalabilidade e manutenção.

Competências técnicas esperadas

Ao final da trilha, o estudante deve dominar:

- SQL avançado.
- Modelagem relacional.
- Modelagem orientada a acesso em NoSQL.
- PostgreSQL avançado.
- Índices e plano de execução.
- Transações, locks, isolamento e deadlocks.
- Backup, restore, PITR e replicação.
- CDC com WAL, Debezium e Kafka.
- MongoDB com modelagem documental, replica set e sharding.
- Redis para cache, rate limiting, locks, streams e ranking.
- DynamoDB com single-table design.
- Busca vetorial com embeddings, pgvector, Qdrant e busca híbrida.
- Observabilidade, segurança, performance e operação de dados.

Classificação por nível

Nível	Conteúdos principais
Básico	SQL, SELECT, JOIN, chaves, constraints, entidades, relacionamentos, índices simples
Intermediário	CTE, subqueries, aggregations, views, transactions, normalização, Redis cache, MongoDB básico
Avançado	Window functions, EXPLAIN ANALYZE, locks, deadlocks, particionamento, CDC, sharding, DynamoDB access patterns
Sênior	Tuning, plano de execução, replicação, backup/PITR, trade-offs de modelagem, arquitetura orientada a eventos, evolução de schema, custo operacional, segurança e escalabilidade

2. Explicação técnica por grande área

2.1 SQL profissional

O que é

SQL profissional é o uso avançado da linguagem SQL para consultar, transformar, agregar, relacionar, proteger e manipular dados com eficiência. Envolve domínio de SELECT, JOINS, CTEs, window functions, subqueries, aggregations, views, materialized views, transações, índices, constraints, query plans e otimização.

No PostgreSQL, EXPLAIN mostra o plano de execução escolhido pelo otimizador, incluindo como tabelas serão escaneadas e quais estratégias de join serão usadas; EXPLAIN ANALYZE executa a query e adiciona métricas reais de execução, o que é essencial para comparar estimativa e realidade.

Para que serve

SQL profissional serve para:

- Buscar dados com precisão.
- Criar relatórios.
- Otimizar consultas críticas.
- Evitar gargalos em APIs.
- Garantir consistência transacional.
- Transformar dados para analytics.
- Reduzir carga desnecessária na aplicação.

Conteúdos essenciais

SELECT avançado

Vai além de SELECT * FROM tabela. Inclui filtros compostos, ordenação eficiente, paginação, projeção seletiva, expressões calculadas, CASE WHEN, DISTINCT ON, funções agregadas e filtros condicionais.

Uso real: listagem de posts, busca de usuários, relatórios financeiros, dashboards administrativos.

Erro comum: buscar colunas demais, usar SELECT * em endpoints públicos, pagnar com OFFSET alto sem estratégia de cursor.

Nível sênior: saber quando uma query aparentemente simples vai degradar com milhões de registros.

JOINS

JOINS conectam tabelas relacionadas. Os principais são INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN, CROSS JOIN e self join.

Uso real: buscar posts com autores, comentários, tags, permissões e métricas.

Problemas que resolve: evita duplicação excessiva e permite reconstruir visões do domínio a partir de tabelas normalizadas.

Erro comum: gerar explosão cartesiana por falta de condição correta de junção.

Nível sênior: entender custo de nested loop, hash join e merge join no plano de execução.

CTE

WITH cria consultas intermediárias reutilizáveis dentro de uma query.

Uso real: pipelines de consulta, relatórios em etapas, consultas recursivas, hierarquias.

Erro comum: usar CTE para tudo sem medir impacto no plano.

Nível sênior: decidir entre CTE, subquery, view, materialized view ou tabela auxiliar.

Window functions

Funções como ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD, SUM() OVER, AVG() OVER.

Uso real: ranking de posts, evolução de receita, comparação com período anterior, top comentários por publicação.

Problema que resolve: permite cálculos por janela sem agrupar e perder detalhe linha a linha.

Nível sênior: usar para analytics sem descarregar dados para a aplicação.

Subqueries

Consultas dentro de consultas.

Uso real: filtros com EXISTS, NOT EXISTS, comparação contra agregações, validação de permissões.

Erro comum: subquery correlacionada mal planejada executando muitas vezes.

Nível sênior: escolher entre subquery, join ou índice parcial com base em plano real.

Aggregations

Uso de COUNT, SUM, AVG, MIN, MAX, GROUP BY, HAVING.

Uso real: dashboards, contagem de comentários, receita por mês, taxa de conversão.

Erro comum: agregar em tabela gigante sem índice, sem particionamento ou sem materialização.

Nível sênior: projetar agregações incrementais e materialized views.

Views e materialized views

Views são consultas salvas. Materialized views armazenam fisicamente o resultado.

Uso real: simplificar consultas complexas, padronizar relatórios, acelerar dashboards.

Trade-off: view comum simplifica leitura, mas não necessariamente melhora performance; materialized view melhora leitura, mas precisa estratégia de refresh.

Transactions

Transações agrupam operações com garantias de atomicidade, consistência, isolamento e durabilidade.

Uso real: criação de pagamento + auditoria + atualização de status.

Erro comum: abrir transações longas, fazer chamada externa dentro da transação, bloquear registros além do necessário.

Nível sênior: definir fronteiras transacionais coerentes com o domínio.

Isolation levels

Controlam como transações simultâneas enxergam dados umas das outras. O PostgreSQL documenta níveis como Read Committed, Repeatable Read e Serializable; a escolha impacta consistência, concorrência e risco de anomalias.

Uso real: evitar inconsistência em pagamentos, estoque, limite de crédito ou processamento financeiro.

Erro comum: assumir que toda leitura é automaticamente consistente em qualquer cenário.

Nível sênior: escolher isolamento conforme criticidade, throughput e risco de conflito.

Locks e deadlocks

Locks protegem dados contra alterações concorrentes conflitantes. Deadlocks ocorrem quando transações esperam recursos umas das outras indefinidamente.

Uso real: atualização de saldo, processamento de pagamentos, alteração de permissões.

Erro comum: atualizar registros em ordens diferentes em fluxos concorrentes.

Nível sênior: reduzir escopo de lock, ordenar operações, usar retry transacional e monitorar contenção.

Indexes

Índices aceleram busca, ordenação e filtros, mas aumentam custo de escrita e ocupam espaço. PostgreSQL oferece vários tipos, como B-tree, Hash, GiST, SP-GiST, GIN e BRIN; B-tree é o padrão e atende casos comuns de igualdade e range queries.

Uso real: busca por usuário, filtros por status, ordenação por data, full-text search, JSONB, geolocalização.

Erro comum: criar índice em tudo, criar índice sem olhar query real, ignorar seletividade.

Nível sênior: desenhar índice a partir de workload, cardinalidade, filtros, ordenação e projeção.

Query plans e EXPLAIN ANALYZE

Plano de execução mostra como o banco pretende executar uma query. EXPLAIN ANALYZE mostra o que aconteceu de fato.

Critério sênior: saber identificar sequential scan indevido, estimativa errada de cardinalidade, nested loop caro, sort em disco, uso ruim de índice, falta de estatística e gargalo de I/O.

Stored procedures e functions

Funções e procedures colocam lógica dentro do banco.

Uso real: validações transacionais, rotinas administrativas, processamento próximo ao dado.

Trade-off: melhora atomicidade e reduz round trips, mas pode acoplar regra de negócio ao banco e dificultar testes/versionamento.

Constraints

Incluem PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, NOT NULL.

Uso real: proteger integridade mesmo se a aplicação falhar.

Erro comum: confiar apenas na validação da aplicação.

Nível sênior: usar constraints como defesa estrutural do domínio.

Normalização e desnormalização controlada

Normalização reduz redundância e anomalias. Desnormalização controlada melhora leitura em cenários específicos.

Uso real: normalizar usuários, posts e tags; desnormalizar contadores, métricas e snapshots de leitura.

Nível sênior: justificar duplicação de dados com base em performance, consistência eventual e custo de manutenção.

2.2 Modelagem relacional

O que é

Modelagem relacional é o processo de representar entidades, atributos, relacionamentos, cardinalidades, regras de integridade e histórico de um domínio em tabelas relacionais.

Para que serve

Serve para transformar regras de negócio em uma estrutura persistente, íntegra, evolutiva e consultável.

Conteúdos fundamentais

Entidades

Representam conceitos do domínio: users, posts, comments, tags, projects, payments.

Erro comum: criar tabela para detalhe técnico sem entender o domínio.

Sênior: modelar a linguagem do negócio, não apenas a estrutura da tela.

Relacionamentos e cardinalidade

Define associações 1:1, 1:N e N:N.

Uso real: um usuário tem muitos posts; post tem muitas tags; comentário pertence a post e usuário.

Erro comum: modelar N:N como string ou array sem avaliar impacto em consulta e integridade.

Chaves naturais vs surrogate keys

Chave natural vem do negócio, como CPF ou e-mail. Surrogate key é artificial, como UUID ou sequence.

Trade-off: chaves naturais têm significado, mas podem mudar; surrogate keys são estáveis, mas exigem constraints adicionais para regras únicas.

Auditoria

Registra criação, alteração, exclusão lógica, origem da mudança e usuário responsável.

Uso real: compliance, troubleshooting, trilha de alteração.

Sênior: definir auditoria técnica e auditoria de negócio separadamente.

Soft delete

Marca registro como excluído sem remover fisicamente.

Uso real: recuperar dados, manter histórico, evitar quebra de relacionamento.

Erro comum: esquecer filtro deleted_at IS NULL, causando vazamento de dados antigos.

Sênior: decidir entre soft delete, hard delete, arquivamento e event sourcing.

Histórico

Armazena versões anteriores de dados.

Uso real: preço histórico, status de pagamento, alterações de perfil.

Sênior: decidir entre tabela histórica, temporal table, eventos de domínio ou snapshot.

Multi-tenancy

Estratégia para atender vários clientes/organizações no mesmo sistema.

Modelos comuns:

- Coluna tenant_id.
- Schema por tenant.
- Banco por tenant.

Trade-off: coluna é mais simples e barata; schema/banco por tenant melhora isolamento, mas aumenta complexidade operacional.

Eventos de domínio

Eventos representam fatos importantes: PostPublished, PaymentApproved, UserRegistered.

Uso real: integração com mensageria, auditoria, notificações, analytics.

Sênior: decidir o que é evento de domínio real e o que é apenas evento técnico.

Modelagem para leitura vs escrita

Escrita prioriza integridade e consistência. Leitura prioriza consulta eficiente.

Uso real: modelo normalizado para transação e projeções/materialized views para dashboard.

Sênior: separar command model e read model quando necessário.

2.3 Projeto prático relacional: banco da DevCore

Modelo sugerido

Entidades principais:

- users
- posts
- tags
- post_tags
- projects
- comments
- analytics_events
- newsletter_subscribers
- payments
- audit_logs

Exemplo de decisões

users deve ter id, email, name, role, status, created_at, updated_at, deleted_at.

posts deve ter id, author_id, title, slug, content, status, published_at, created_at, updated_at.

tags deve ter id, name, slug.

post_tags resolve N:N entre posts e tags.

payments deve guardar user_id, provider, amount, currency, status, external_reference, paid_at.

audit_logs deve guardar actor_id, entity_type, entity_id, action, before_data, after_data, created_at.

Critério sênior do projeto

O modelo precisa responder bem a perguntas como:

- Como garantir slug único por post?
- Como consultar posts publicados por tag?
- Como auditar mudança de status?
- Como manter histórico de pagamento?
- Como isolar dados por tenant?
- Como evitar perda de evento após commit?
- Como paginar posts sem degradar com milhões de linhas?
- Como proteger dados sensíveis?
- Como restaurar dados após falha?

2.4 PostgreSQL avançado

O que é

PostgreSQL avançado envolve uso profundo de índices, particionamento, estatísticas, VACUUM, replicação, WAL, backup, restore, pool de conexões e diagnóstico operacional.

Índices avançados

B-tree

Ideal para igualdade, ordenação e range queries.

Uso real: WHERE user_id = ?, ORDER BY created_at DESC.

GIN

Muito usado para arrays, JSONB e full-text search.

Uso real: busca por tags, permissões e campos JSONB.

GIST

Usado em tipos geométricos, ranges e alguns cenários especializados.

BRIN

Bom para tabelas grandes onde a ordem física dos dados tem correlação com o valor, como logs por data.

Partial indexes

Índice apenas em parte dos dados.

Exemplo: posts publicados.

```
CREATE INDEX idx_posts_published
ON posts (published_at DESC)
WHERE status = 'PUBLISHED';
```

Expression indexes

Índice baseado em expressão.

Exemplo:

```
CREATE INDEX idx_users_lower_email
ON users (lower(email));
```

- Covering indexes

Índices que incluem colunas suficientes para evitar acesso adicional à tabela.

Sênior: escolher índice não pelo campo isolado, mas pela query completa: filtros, ordenação, projeção, cardinalidade e frequência.

Partitioning

Particionamento divide uma tabela grande em partes menores.

Tipos:

- Range partitioning.
- List partitioning.
- Hash partitioning.
- Foreign partitions.
- Partition pruning.

Uso real: analytics_events particionada por mês; payments por data; logs por tenant.

Problema que resolve: melhora manutenção, limpeza, retenção, leitura por intervalo e gerenciamento de tabelas muito grandes.

Erro comum: particionar cedo demais ou escolher chave de partição que não aparece nos filtros.

Sênior: definir partição com base em volume, retenção, padrão de consulta, custo de manutenção e operação de backup.

VACUUM, AUTOVACUUM, ANALYZE, bloat e dead tuples

PostgreSQL usa MVCC; atualizações e deleções podem deixar versões antigas de linhas, chamadas dead tuples. VACUUM e autovacuum ajudam a recuperar espaço reutilizável e manter o banco saudável; estatísticas atualizadas também são essenciais para o otimizador escolher bons planos.

Uso real: tabelas com muitas atualizações, filas no banco, logs, pagamentos, sessões.

Erro comum: desabilitar autovacuum, ignorar bloat ou não monitorar tabelas com alto write rate.

Sênior: ajustar autovacuum por tabela crítica, monitorar dead tuples, bloat, freeze e impacto em performance.

Statistics e pg_stat_statements

Estatísticas ajudam o otimizador a estimar cardinalidade e custo. pg_stat_statements é usado para identificar queries frequentes, lentas ou custosas.

Uso real: descobrir endpoints que causam maior carga no banco.

Sênior: tomar decisão com evidência: p95/p99, frequência, tempo total, rows scanned, buffers e I/O.

Replication, WAL e recuperação

WAL é o mecanismo de log de alterações usado pelo PostgreSQL para integridade e recuperação. Ele registra mudanças antes de aplicá-las definitivamente aos arquivos de dados, permitindo recuperação após crash e suporte a backup online/PITR.

Streaming replication

Replica alterações físicas para standby.

Uso real: alta disponibilidade e leitura em réplicas.

Logical replication

Replica mudanças em nível lógico, como tabelas/publicações, útil para integração e migração.

Replication slots

Mantém WAL necessário para consumidores.

Risco: slot parado pode acumular WAL e encher disco.

pg_basebackup

Cria base backup física.

PITR

Point-in-time recovery permite restaurar o banco para um instante específico combinando base backup e WAL arquivado.

Switchover e failover

Switchover: troca planejada.

Failover: troca emergencial após falha.

Sênior: saber desenhar RPO/RTO, testar restore, documentar runbook e evitar falsa sensação de segurança.

PgBouncer e pgpool-II

Ferramentas de pooling/proxy.

PgBouncer: reduz overhead de conexões.

pgpool-II: pode oferecer pooling, balanceamento e recursos adicionais.

Trade-off: pooling transaction mode pode afetar prepared statements, session variables e alguns comportamentos de sessão.

2.5 CDC e arquitetura orientada a eventos

O que é

CDC, Change Data Capture, é a captura de alterações do banco para publicação em outro sistema, geralmente Kafka. Debezium é uma plataforma de CDC que monitora bancos e permite que aplicações respondam a inserts, updates e deletes publicados como eventos.

Componentes

WAL

Fonte das mudanças no PostgreSQL.

Logical decoding

Transforma WAL em mudanças lógicas compreensíveis.

Debezium

Conector que lê mudanças e publica eventos.

Kafka Connect

Infraestrutura para rodar conectores.

Outbox via CDC

Padrão onde a aplicação grava dados de negócio e evento na mesma transação. O Debezium lê a tabela outbox e publica no Kafka.

Problemas que resolve

Evita dual write inconsistente.

Reduz polling.

Permite integração assíncrona.

Desacopla serviços.

Alimenta analytics, busca, notificações e auditoria.

Erros comuns

Publicar evento antes do commit.

Não tratar idempotência.

Ignorar ordering.

Não versionar schema.

Não planejar reprocessamento.

Não monitorar lag do conector.

Não tratar tombstone/delete corretamente.

Critério sênior

Saber decidir entre:

- Evento produzido pela aplicação.
- Outbox transacional.
- CDC direto de tabela.
- Event sourcing.
- Sincronização batch.
- Integração síncrona via API.

Projeto prático

Fluxo:

PostgreSQL → Debezium → Kafka Connect → Kafka → Consumer Java/Python

Eventos:

- UserCreated
- PostPublished
- PostUpdated
- PaymentApproved
- PaymentFailed

Critérios:

- Evento deve ter event_id, aggregate_id, event_type, payload, schema_version, occurred_at.
- Consumer deve ser idempotente.
- Deve haver DLQ.
- Deve haver observabilidade de lag.
- Deve haver replay controlado.

2.6 MongoDB avançado

O que é

MongoDB é um banco documental. A modelagem é orientada a documentos, não a tabelas. MongoDB suporta workloads como busca documental, agregações, full-text search, geospatial, time series e outros cenários modernos de aplicação.

Document modeling

Modelagem documental define como dados serão agrupados em documentos.

Embedding vs referencing

Embedding

Coloca dados relacionados dentro do mesmo documento.

Bom para: leitura conjunta, dados pequenos, ciclo de vida comum.

Exemplo: comentários curtos embutidos em post pequeno.

Referencing

Guarda referência entre documentos.

Bom para: dados grandes, muitos relacionamentos, crescimento ilimitado.

Exemplo: comentários como coleção separada.

Índices

MongoDB suporta índices para melhorar performance de consultas; o uso correto depende dos filtros, ordenação e cardinalidade da consulta.

Tipos importantes:

- Single field.
- Compound index.
- TTL index.
- Text index.
- Sparse/partial indexes.

Aggregation framework

Permite pipelines de transformação e agregação.

Uso real: analytics, relatórios, agrupamentos, enriquecimento de dados.

Transactions

MongoDB suporta transações distribuídas em cenários que exigem atomicidade entre múltiplos documentos, coleções ou bancos, inclusive em replica sets e sharded clusters.

Trade-off: transação existe, mas não deve ser usada para compensar modelagem documental ruim.

Replica sets

Conjunto de nós para alta disponibilidade.

Sharding

Distribui dados horizontalmente. MongoDB usa shard key para distribuir documentos entre shards; a escolha da shard key é uma das decisões mais críticas de escalabilidade.

Shard key design

Uma boa shard key deve considerar:

- Distribuição uniforme.
- Alta cardinalidade.
- Frequência de acesso.
- Evitar monotonicidade extrema.
- Evitar hotspots.
- Suportar queries frequentes.

Erros comuns

Usar campo incremental como shard key.

Criar documento que cresce indefinidamente.

Não criar índices alinhados com queries.

Usar MongoDB como se fosse relacional.

Fazer join mental demais na aplicação.

Ignorar tamanho máximo e crescimento de arrays.

Critério sênior

Saber responder:

- Esse dado deve ser embutido ou referenciado?
- Qual é o padrão de leitura dominante?
- O documento pode crescer sem limite?
- Qual índice atende a query real?
- A shard key distribui carga ou cria hotspot?
- O custo de transação é aceitável?
- O dado precisa mesmo estar no MongoDB ou deveria estar no PostgreSQL?

2.7 Redis avançado

O que é

Redis é um servidor de estruturas de dados em memória, usado para cache, filas leves, ranking, sessões, locks, rate limiting e processamento de eventos. Redis oferece tipos como strings, hashes, lists, sets,

sorted sets e streams.

Estruturas principais

Strings

Usadas para cache simples, contadores e flags.

Hashes

Representam objetos pequenos.

Uso real: perfil de usuário cacheado.

Lists

Filas simples.

Sets

Conjuntos únicos.

Uso real: permissões, usuários online.

Sorted sets

Conjuntos ordenados por score.

Uso real: ranking, leaderboard, trending posts.

Streams

Redis Streams funcionam como log append-only para eventos e permitem estratégias de consumo como consumer groups.

Uso real: fila leve, eventos internos, processamento assíncrono simples.

Pub/Sub

Mensagens em tempo real sem persistência forte.

Uso real: notificações efêmeras.

TTL e eviction policies

TTL define expiração. Eviction define o que remover quando a memória fica cheia.

Erro comum: usar Redis sem TTL em cache.

Sênior: definir política conforme criticidade: cache pode expirar; sessão talvez exija persistência/replicação; lock precisa TTL curto e seguro.

Padrões de cache

Cache aside

Aplicação consulta cache; se não existir, busca no banco e preenche.

Write-through

Escrita passa pelo cache e persistência.

Write-behind

Escrita assíncrona posterior.

Trade-off: cache aside é simples; write-through melhora consistência; write-behind aumenta risco de perda.

Distributed locks e Redlock

Redis possui documentação sobre algoritmo Redlock para locks distribuídos, mas esse tema deve ser tratado com cautela: lock distribuído exige análise de falha, tempo, relógio, rede e idempotência.

Redis Cluster

Redis Cluster distribui dados horizontalmente por hash slots; a documentação descreve 16.384 hash slots e sharding automático entre nós.

Problemas comuns

Hot keys.

Hot slots.

Cache stampede.

Cache penetration.

Cache avalanche.

Falta de TTL.

Dados sensíveis em cache sem proteção.

Uso de Redis como banco primário sem estratégia adequada.

Projeto prático

Implementar na DevCore:

- Cache de posts publicados.
- Rate limiter por IP/usuário.
- Ranking de posts com sorted set.
- Lock distribuído para job crítico.
- Fila leve com Redis Streams.
- Sessão de usuário com TTL.

Critério sênior

Saber quando Redis é cache, quando é fila, quando é coordenação e quando não deve ser usado.

2.8 DynamoDB

O que é

DynamoDB é um banco NoSQL gerenciado da AWS baseado em tabelas, itens e atributos. Ele usa chave primária para identificar itens e pode usar Streams para capturar eventos de modificação.

Partition key e sort key

A partition key define a partição física via hash interno. Com sort key, múltiplos itens podem compartilhar a mesma partition key e ficam ordenados pela sort key.

Single-table design

Modelagem onde várias entidades ficam na mesma tabela, organizadas por padrões de acesso.

Exemplo:

```
PK = USER#123
SK = PROFILE
```

```
PK = USER#123
SK = POST#2026-06-01#999
```

PK = POST#999
SK = COMMENT#001

GSIs e LSIs

GSI permite consultar com chave diferente da tabela.

LSI mantém a mesma partition key, mas muda sort key.

DynamoDB mantém índices secundários automaticamente, mas eles afetam custo, escrita, projeção e capacidade.

TTL

Remove itens expirados.

Uso real: sessões, tokens, dados temporários.

Streams

DynamoDB Streams captura modificações item-level em ordem temporal próxima do tempo real e pode ser usado para integrações, materialized views e triggers com Lambda.

Conditional writes

Permite escrita condicional.

Uso real: evitar sobrescrever versão mais nova, implementar concorrência otimista.

Transactions

Usadas quando múltiplos itens precisam ser alterados atomicamente.

Hot partitions

Ocorrem quando uma partition key concentra leituras/escritas demais.

On-demand vs provisioned

On-demand: simplicidade e elasticidade.

Provisioned: controle de custo quando tráfego é previsível.

Global tables

Replicação multi-região.

DAX

Cache gerenciado para DynamoDB.

Critério sênior

Modelar DynamoDB a partir de access patterns, não a partir de entidade relacional.

Perguntas obrigatórias:

- Quais queries precisam ser respondidas?

- Qual PK/SK atende cada acesso?
- Precisa de GSI?
- Existe risco de hot partition?
- Qual item collection pode crescer demais?
- Qual consistência é exigida?
- Qual custo por leitura/escrita?
- O dado precisa estar em DynamoDB ou PostgreSQL resolveria melhor?

2.9 Vector databases e busca semântica

O que é

Busca vetorial usa embeddings para representar textos, documentos ou entidades como vetores numéricos. A busca retorna itens semanticamente próximos, não apenas textos com palavras iguais.

pgvector é uma extensão PostgreSQL para armazenar embeddings e executar busca por similaridade vetorial dentro do Postgres; ela suporta índices como HNSW e IVFFlat, com trade-offs de build time, memória, velocidade e recall.

Conceitos principais

Embeddings

Representações vetoriais de texto, imagem ou dados.

Similaridade cosseno

Mede proximidade angular entre vetores.

Dot product

Usado quando magnitude também tem significado.

Euclidean distance

Mede distância geométrica.

ANN

Approximate Nearest Neighbor, busca aproximada para acelerar consulta em grande volume.

HNSW

Índice baseado em grafo, geralmente bom para alta performance e bom recall, com maior custo de memória.

IVF

Divide vetores em listas/clusters para reduzir espaço de busca.

Hybrid search

Combina busca lexical e vetorial.

Exemplo:

BM25 / PostgreSQL FTS + pgvector + filtros + reranking

Qdrant oferece recursos para busca vetorial, filtros e consultas híbridas, incluindo estratégias de reranking e combinação de índices vetoriais com filtros estruturados.

Chunking

Divisão de documentos em partes menores para indexação.

Erro comum: chunks grandes demais perdem precisão; chunks pequenos demais perdem contexto.

Metadata filtering

Filtros por tag, permissão, autor, tenant, status.

Uso real: buscar apenas conteúdos que o usuário tem permissão para acessar.

Reranking

Reordenação dos resultados com modelo mais caro ou regra de negócio.

Projeto prático: busca híbrida da DevCore

Arquitetura:

posts → chunking → embeddings → pgvector posts/tags/status → PostgreSQL FTS consulta → lexical + vetorial → filtro por permissão/tag → reranking → resultado

Critérios:

- Respeitar permissões.
- Filtrar por status publicado.
- Filtrar por tenant.
- Medir recall.
- Medir latência p95.
- Reindexar ao alterar conteúdo.
- Versionar embeddings.

Critério sênior

Saber decidir entre:

- PostgreSQL FTS puro.
- pgvector.
- Qdrant.
- Elasticsearch/OpenSearch.
- Busca híbrida.
- RAG com reranking.

3. Organização como plano de estudos

Módulo 1 — Fundamentos de dados e SQL

Objetivo

Construir base sólida em SQL, modelagem relacional e integridade.

Conteúdos

- Entidades, atributos e relacionamentos.
- Cardinalidade.
- Chaves primárias e estrangeiras.
- Constraints.
- SELECT.
- JOINS.
- Aggregations.
- Subqueries.
- Normalização.
- Transações básicas.

Conceitos obrigatórios

- 1FN, 2FN, 3FN.
- PK, FK, UNIQUE, CHECK, NOT NULL.
- INNER JOIN e LEFT JOIN.
- GROUP BY e HAVING.
- Atomicidade.
- Consistência.
- Relacionamento N:N.

Práticas recomendadas

- Nunca modelar sem entender access patterns.
- Evitar SELECT *.
- Criar constraints no banco.
- Nomear tabelas e colunas de forma consistente.
- Criar migrations versionadas.

Exercícios técnicos

- Criar banco de usuários, posts e comentários.
- Escrever queries com joins.
- Criar constraints de unicidade.
- Criar query de posts por tag.
- Criar relatório de posts por mês.

Mini projeto

Modelar núcleo inicial da DevCore:

- Usuários.
- Posts.
- Comentários.
- Tags.

Critério de conclusão

Você conclui o módulo quando conseguir criar o schema, popular dados e responder consultas reais sem depender de ORM.

Nível esperado

Básico sólido caminhando para intermediário.

Módulo 2 — SQL avançado e aplicação prática

Objetivo

Dominar consultas complexas e análise de dados transacionais.

Conteúdos

- CTE.
- Window functions.
- Views.
- Materialized views.
- Stored functions.
- Stored procedures.
- Paginação eficiente.
- Índices básicos.
- Transações.
- Isolation levels.

Conceitos obrigatórios

- ROW_NUMBER.
- RANK.
- LAG / LEAD.
- EXISTS.
- Cursor pagination.
- Transaction boundary.
- Read committed vs serializable.
- Idempotência básica.

Práticas recomendadas

- Medir queries antes/depois.
- Usar materialized views para leitura pesada.
- Criar índices alinhados com queries.
- Evitar transação longa.
- Não fazer chamada HTTP dentro da transação.

Exercícios técnicos

- Ranking de posts mais acessados.
- Receita acumulada por mês.
- Posts com crescimento semanal.
- Comentários recentes por usuário.
- Paginação por cursor.

Mini projeto

Dashboard SQL da DevCore:

- Posts publicados por período.
- Top tags.
- Usuários mais ativos.
- Receita por mês.

Conversão newsletter → pagamento.

Critério de conclusão

Conseguir escrever query analítica com window function e explicar o plano de execução.

Nível esperado

Intermediário forte.

Módulo 3 — PostgreSQL avançado e performance

Objetivo

Aprender a diagnosticar, otimizar e operar PostgreSQL.

Conteúdos

- B-tree, GIN, GiST, BRIN.
- Partial indexes.
- Expression indexes.
- Covering indexes.
- EXPLAIN ANALYZE.
- Query plans.
- Statistics.
- VACUUM.

- AUTOVACUUM.
- Bloat.
- Dead tuples.
- Partitioning.
- pg_stat_statements.
- pg_locks.
- pg_activity.

Conceitos obrigatórios

- Sequential scan.
- Index scan.
- Bitmap index scan.
- Hash join.
- Nested loop.
- Sort em memória vs disco.
- Cardinalidade estimada vs real.
- Buffer hit/read.
- Dead tuple.
- Partition pruning.

Práticas recomendadas

- Usar EXPLAIN (ANALYZE, BUFFERS).
- Monitorar queries por tempo total, não apenas por tempo médio.
- Criar índice somente com hipótese clara.
- Remover índice inútil.
- Testar particionamento com massa real.
- Medir impacto de escrita.

Exercícios técnicos

- Criar query lenta propositalmente.
- Otimizar com índice correto.
- Comparar B-tree e GIN.
- Criar tabela particionada por mês.
- Simular bloat com updates.
- Monitorar locks.

Mini projeto

Tuning da DevCore:

- Criar 1 milhão de eventos analytics.
- Particionar por mês.
- Criar índices parciais.

- Otimizar dashboard.
- Documentar antes/depois.

Critério de conclusão

Conseguir identificar o gargalo de uma query lenta por plano de execução, não por chute.

Nível esperado

Avançado.

Módulo 4 — Arquitetura de dados e eventos

Objetivo

Integrar banco de dados com arquitetura orientada a eventos.

Conteúdos

- WAL.
- Logical decoding.
- Debezium.
- Kafka Connect.
- Outbox pattern.
- Ordering.
- Idempotência.
- Reprocessamento.
- Schema evolution.
- Data contracts.
- Event sourcing conceitual.

Conceitos obrigatórios

- Dual write problem.
- At-least-once delivery.
- Consumer idempotente.
- Event versioning.
- DLQ.
- Replay.
- Outbox transacional.
- Lag de replicação.
- Schema Registry.

Práticas recomendadas

- Evento precisa ter ID único.
- Consumer precisa tolerar duplicidade.
- Schema precisa ser versionado.
- Replay precisa ser testado.

- DLQ precisa ter processo de tratamento.
- Não publicar evento fora da transação sem estratégia.

Exercícios técnicos

- Criar tabela outbox.
- Publicar eventos via Debezium.
- Consumir eventos com Java.
- Simular consumer duplicado.
- Simular falha e replay.
- Versionar payload.

Mini projeto

CDC da DevCore:

PostgreSQL → Debezium → Kafka → Consumers

Eventos:

- Usuário criado.
- Post publicado.
- Pagamento aprovado.
- Comentário criado.

Critério de conclusão

Conseguir explicar como evita perda de evento entre commit no banco e publicação no Kafka.

Nível esperado

Avançado com base arquitetural.

Módulo 5 — NoSQL, cache, escalabilidade e busca

Objetivo

Aprender quando e como usar MongoDB, Redis, DynamoDB e bancos vetoriais.

Conteúdos

- MongoDB document modeling.
- Embedding vs referencing.
- MongoDB indexes.
- Aggregation framework.
- Replica sets.
- Sharding.
- Redis data structures.
- Cache aside.
- Redis Streams.
- Rate limiting.

- Distributed locks.
- DynamoDB single-table design.
- GSIs/LSIs.
- DynamoDB Streams.
- Vector search.
- pgvector.
- Qdrant.
- Hybrid search.

Conceitos obrigatórios

- Access pattern.
- Shard key.
- Hotspot.
- Hot key.
- TTL.
- Eviction.
- Cache invalidation.
- Partition key.
- Sort key.
- ANN.
- HNSW.
- Reranking.
- Metadata filtering.

Práticas recomendadas

- Não usar NoSQL sem access pattern.
- Não usar Redis como banco primário sem justificativa.
- Não usar DynamoDB como banco relacional.
- Não usar busca vetorial sem avaliação de qualidade.
- Não ignorar permissões em busca semântica.
- Medir latência p95/p99.

Exercícios técnicos

- Criar ranking com Redis sorted set.
- Criar rate limiter.
- Criar fila com Redis Streams.
- Modelar posts no MongoDB.
- Escolher shard key para analytics.
- Modelar tabela única no DynamoDB.
- Criar busca híbrida com pgvector.

Mini projeto

Camada de performance da DevCore:

- PostgreSQL como banco transacional.
- Redis para cache/rate limit.
- MongoDB para conteúdo flexível ou logs enriquecidos, se houver justificativa.
- DynamoDB para access pattern específico de alta escala.
- pgvector/Qdrant para busca semântica.

Critério de conclusão

Conseguir justificar por que cada banco existe na arquitetura e qual problema ele resolve.

Nível esperado

Avançado.

Módulo 6 — Domínio sênior, produção e liderança técnica

Objetivo

Formar visão de decisão técnica, operação, segurança, custo, evolução e liderança.

Conteúdos

- Backup e restore.
- PITR.
- Replicação.
- Failover.
- Switchover.
- Pool de conexão.
- Observabilidade.
- Segurança.
- LGPD/dados sensíveis.
- Criptografia.
- Retenção.
- Migração.
- Estratégia multi-tenant.
- Estratégia de evolução de schema.
- Revisão técnica.
- ADRs.
- Runbooks.
- Postmortems.

Conceitos obrigatórios

- RPO.
- RTO.

- Least privilege.
- Encryption at rest/in transit.
- Connection pool.
- Slow query log.
- Capacity planning.
- Disaster recovery.
- Data lifecycle.
- Zero-downtime migration.
- Backward compatibility.

Práticas recomendadas

- Testar restore, não apenas fazer backup.
- Criar runbook de incidente.
- Medir custo de índices e replicação.
- Usar migrations reversíveis quando possível.
- Fazer rollout gradual de mudanças de schema.
- Revisar queries críticas em PR.
- Documentar decisões arquiteturais.

Exercícios técnicos

- Simular falha e restore.
- Criar plano de backup.
- Criar matriz RPO/RTO.
- Fazer migração sem downtime.
- Criar dashboard de observabilidade.
- Criar ADR de escolha entre PostgreSQL, MongoDB e DynamoDB.

Mini projeto

Operação sênior da DevCore:

- Backup com teste de restore.
- Replica read-only.
- PgBouncer.
- Monitoramento de locks.
- Alertas de WAL, bloat e conexão.
- Runbook de incidente.
- ADRs de decisões de banco.

Critério de conclusão

Conseguir defender a arquitetura de dados em uma revisão técnica sênior, explicando trade-offs, riscos, custo, evolução e impacto em produção.

Nível esperado

Sênior inicial em dados aplicados a backend/arquitetura.

4. Critérios de domínio técnico por área

SQL e modelagem relacional

Nível	Critério
Iniciante	Entende tabelas, chaves, SELECT, JOIN básico e constraints
Intermediário	Modela entidades, cria queries com agregações, usa transações e normalização
Avançado	Usa CTE, window functions, views, índices e lê planos de execução
Sênior	Decide normalização vs desnormalização, otimiza queries críticas, revisa schema e previne problemas de produção

PostgreSQL

Nível	Critério
Iniciante	Cria tabelas, índices simples e consultas básicas
Intermediário	Usa migrations, constraints, transações e índices adequados
Avançado	Diagnostica queries, locks, bloat, autovacuum e particionamento
Sênior	Opera backup, restore, replicação, failover, tuning e capacidade em produção

CDC e eventos

Nível	Critério
Iniciante	Entende evento, fila e tópico
Intermediário	Publica/consume eventos com Kafka
Avançado	Implementa outbox, idempotência, DLQ e replay
Sênior	Decide modelo de integração, schema evolution, ordering, contrato e estratégia de reprocessamento

MongoDB

Nível	Critério
Iniciante	Entende documentos e coleções
Intermediário	Cria índices e pipelines simples
Avançado	Modela embedding/reference, usa transactions e replica set
Sênior	Define shard key, evita hotspots, revisa crescimento de documento e decide se MongoDB é adequado

Redis

Nível	Critério
Iniciante	Usa Redis como cache simples
Intermediário	Usa TTL, hashes, sets e sorted sets
Avançado	Implementa rate limit, streams, locks e cluster
Sênior	Decide estratégia de cache, invalidação, alta disponibilidade, hot keys e risco de consistência

DynamoDB

Nível	Critério
Iniciante	Entende tabela, item e chave primária
Intermediário	Usa PK/SK, GSI, TTL e conditional writes
Avançado	Modela single-table design por access patterns
Sênior	Evita hot partitions, estima custo, projeta multi-região e justifica consistência/capacidade

Busca vetorial

Nível	Critério
Iniciante	Entende embeddings e similaridade
Intermediário	Implementa busca com pgvector
Avançado	Usa HNSW/IVF, filtros e reranking
Sênior	Decide arquitetura de busca híbrida, mede recall/latência e protege permissões no retrieval

5. Aplicação prática em sistemas reais

APIs

Banco impacta diretamente latência, paginação, consistência e escalabilidade.

Exemplo:

```
GET /posts?tag=java&cursor=...
```

Decisões:

- Índice em status, published_at, tag_id.
- Cursor pagination.
- Cache Redis para posts populares.
- Filtro por permissão.
- Observabilidade da query.

Banco de dados

PostgreSQL deve concentrar dados transacionais fortes: usuários, posts, pagamentos, auditoria e permissões.

Microserviços

Cada serviço deve ser dono dos seus dados. Evite banco compartilhado sem governança.

Exemplo:

- ms-users
- ms-content
- ms-payments
- ms-analytics

Mensageria

Eventos propagam mudanças.

Exemplo:

PaymentApproved → liberar acesso premium PostPublished → indexar busca UserCreated → enviar onboarding

Segurança

Aplicar:

- Least privilege.
- Criptografia em trânsito.
- Controle de acesso por schema/tabela.
- Mascaramento de dados sensíveis.
- Auditoria.
- Retenção e expurgo.

Observabilidade

Monitorar:

- Slow queries.
- Locks.
- Deadlocks.
- Conexões.
- Cache hit ratio.
- Replication lag.
- WAL growth.
- Bloat.
- Latência p95/p99.
- Erros de consumer.

Testes

Tipos relevantes:

- Testes de repository.
- Testes de migration.
- Testes com Testcontainers.
- Testes de concorrência.
- Testes de idempotência.
- Testes de restore.
- Testes de contrato de evento.

Deploy

Cuidados:

- Migrations backward-compatible.
- Não renomear coluna diretamente sem fase intermediária.
- Evitar lock longo em tabela grande.
- Rodar índices concorrentes quando aplicável.
- Separar deploy de aplicação e migração destrutiva.

Performance

Estratégias:

- Índices adequados.
- Paginação por cursor.
- Cache.
- Materialized views.
- Particionamento.
- Read replicas.
- Batch processing.
- Compressão/arquivamento.

Manutenção evolutiva

Toda mudança de schema deve responder:

- É compatível com versão anterior da aplicação?
- Tem rollback?
- Impacta dados existentes?
- Precisa backfill?
- Pode bloquear tabela?
- Como será monitorada?

6. Cronograma sugerido de execução

Visão geral

Sugestão: 24 semanas, com prática contínua e projeto DevCore evoluindo ao longo da trilha.

Período	Foco	Resultado
Semanas 1-3	SQL e modelagem	Schema relacional inicial
Semanas 4-6	SQL avançado	Queries analíticas e views
Semanas 7-10	PostgreSQL performance	Tuning com EXPLAIN ANALYZE
Semanas 11-13	Operação PostgreSQL	Backup, restore, replicação, PITR
Semanas 14-16	CDC e Kafka	Outbox + Debezium + consumidor
Semanas 17-18	MongoDB	Modelagem documental e sharding conceitual
Semanas 19-20	Redis	Cache, rate limit, ranking e streams
Semanas 21-22	DynamoDB	Single-table design por access pattern
Semanas 23-24	Busca vetorial	pgvector, busca híbrida e reranking

Checkpoints técnicos

Checkpoint 1 — Semana 3

Entregar:

- DER da DevCore.
- Migrations.
- Constraints.
- Queries básicas.
- Explicação de normalização.

Checkpoint 2 — Semana 6

Entregar:

- Queries com CTE.
- Window functions.
- Views/materialized views.
- Relatórios de negócio.

Checkpoint 3 — Semana 10

Entregar:

- Query lenta antes/depois.
- Plano com EXPLAIN ANALYZE.
- Índices justificados.

- Tabela particionada.
- Métricas comparativas.

Checkpoint 4 — Semana 13

Entregar:

- Estratégia de backup.
- Teste de restore.
- Plano RPO/RTO.
- Simulação de failover.
- Runbook operacional.

Checkpoint 5 — Semana 16

Entregar:

- Outbox pattern.
- Debezium.
- Kafka.
- Consumer idempotente.
- DLQ.
- Replay documentado.

Checkpoint 6 — Semana 20

Entregar:

- Cache Redis.
- Rate limiter.
- Ranking.
- Redis Streams.
- Modelagem MongoDB com justificativa.

Checkpoint 7 — Semana 22

Entregar:

- Tabela DynamoDB.
- Access patterns.
- PK/SK.
- GSI.
- Simulação de hot partition.
- Estimativa de custo.

Checkpoint 8 — Semana 24

Entregar:

- Busca híbrida.
- pgvector.
- FTS.
- Reranking.
- Filtro por permissão.
- Avaliação de recall e latência.

7. Projetos recomendados para portfólio

Projeto 1 — DevCore Database Platform

Sistema completo com:

- PostgreSQL.
- Modelagem relacional.
- Pagamentos.
- Posts.
- Tags.
- Comentários.
- Auditoria.
- Newsletter.
- Analytics.

Diferencial: mostrar DER, migrations, constraints e decisões técnicas.

Projeto 2 — PostgreSQL Performance Lab

Projeto focado em:

- Query lenta.
- EXPLAIN ANALYZE.
- Índices.
- Particionamento.
- Autovacuum.
- pg_stat_statements.
- Benchmark antes/depois.

Diferencial: documentação técnica com evidências.

Projeto 3 — CDC Event Pipeline

Projeto com:

- PostgreSQL.
- Outbox.

- Debezium.
- Kafka.
- Consumer Java/Python.
- DLQ.
- Replay.

Diferencial: resolver dual write problem.

Projeto 4 — Redis Performance Layer

Projeto com:

- Cache aside.
- Rate limiter.
- Ranking.
- Lock distribuído.
- Redis Streams.

Diferencial: mostrar trade-offs de consistência e invalidação.

Projeto 5 — Hybrid Semantic Search

Projeto com:

- PostgreSQL FTS.
- pgvector.
- Embeddings.
- Filtros por permissão.
- Reranking.
- Métricas de busca.

Diferencial: aplicar IA/busca semântica com segurança e arquitetura real.

8. Checklist de domínio técnico

SQL e modelagem

I Sei modelar entidades, relacionamentos e cardinalidade.

I Sei normalizar e desnormalizar com justificativa.

I Sei escrever JOINS complexos.

I Sei usar CTE e window functions.

I Sei criar views e materialized views.

I Sei trabalhar com transações.

I Sei explicar isolation levels.

I Sei evitar deadlocks comuns.

PostgreSQL

I Sei ler EXPLAIN ANALYZE.

I Sei identificar query lenta.

I Sei escolher índice adequado.

I Sei diferenciar B-tree, GIN, GiST e BRIN.

I Sei usar partial/expression/covering indexes.

I Sei explicar VACUUM, AUTOVACUUM e bloat.

I Sei particionar tabela grande.

I Sei monitorar locks e queries.

I Sei planejar backup, restore e PITR.

I Sei explicar replicação e failover.

CDC e eventos

I Sei explicar WAL e logical decoding.

I Sei implementar outbox.

I Sei configurar Debezium conceitualmente.

I Sei criar consumer idempotente.

I Sei lidar com replay.

I Sei versionar contrato de evento.

I Sei explicar ordering e duplicidade.

MongoDB

I Sei decidir embedding vs referencing.

I Sei criar compound indexes.

I Sei usar aggregation framework.

I Sei explicar replica set.

I Sei projetar shard key.

I Sei evitar documento de crescimento ilimitado.

I Sei identificar antipadrões.

Redis

I Sei usar strings, hashes, sets, sorted sets e streams.

I Sei aplicar TTL.

I Sei definir estratégia de cache.

I Sei implementar rate limiter.

- I Sei lidar com hot keys.
- I Sei explicar Redis Cluster.
- I Sei discutir lock distribuído com cautela.

DynamoDB

- I Sei modelar por access patterns.
- I Sei usar PK/SK.
- I Sei projetar GSIs.
- I Sei evitar hot partitions.
- I Sei usar conditional writes.
- I Sei explicar TTL, Streams e capacity modes.
- I Sei justificar single-table design.

Busca vetorial

- I Sei explicar embeddings.
- I Sei diferenciar cosine, dot product e Euclidean distance.
- I Sei usar pgvector.
- I Sei explicar HNSW e IVF.
- I Sei fazer busca híbrida.
- I Sei aplicar filtros por permissão.
- I Sei medir qualidade e latência.

9. Indicadores de prontidão para nível avançado/sênior

Você está pronto para atuar em nível avançado/sênior nessa trilha quando conseguir:

- Analisar uma query lenta em produção e propor correção baseada em plano de execução.
- Modelar um banco relacional completo com integridade, auditoria, histórico e multi-tenancy.
- Decidir entre PostgreSQL, MongoDB, Redis e DynamoDB com justificativa técnica.
- Projetar estratégia de cache considerando invalidação, TTL, consistência e hot keys.
- Implementar CDC com outbox sem risco de dual write.
- Definir backup, restore, RPO e RTO para um sistema real.
- Explicar impactos de particionamento, índices e replicação em custo e operação.
- Criar busca híbrida com segurança, respeitando permissões e filtros.
- Revisar PRs de modelagem e queries apontando riscos reais de produção.
- Documentar decisões arquiteturais com trade-offs, riscos, alternativas e plano de evolução.

10. Resumo final da trilha

Esta trilha forma domínio completo em dados para backend, arquitetura e sistemas distribuídos. O estudante começa com SQL e modelagem relacional, evolui para PostgreSQL avançado, passa por CDC e arquitetura orientada a eventos, aprofunda NoSQL com MongoDB, Redis e DynamoDB, e finaliza com busca vetorial e visão sênior de produção.

O objetivo final é que o profissional não apenas “use banco de dados”, mas consiga projetar a estratégia de dados de um sistema real, justificando decisões de modelagem, performance, segurança, escalabilidade, custo e manutenção.